

MINI GITHUB

A PROJECT REPORT

Submitted by

BALAMANIKANDAN S

in partial fulfilment for the award of the course

CGB1221 – DATABASE MANAGEMENT SYSTEMS

IN

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)



**K. RAMAKRISHNAN COLLEGE OF ENGINEERING
(AUTONOMOUS)
SAMAYAPURAM, TRICHY**



**ANNA UNIVERSITY
CHENNAI 600 025**

MAY 2026

MINI GITHUB

PROJECT FINAL DOCUMENT

Submitted by

BALAMANIKANDAN S (8115U24AM013)

in partial fulfilment for the award of the course

CGB1221 – DATABASE MANAGEMENT SYSTEMS

IN

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Under the Guidance of

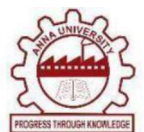
Mr. P. NAGARAJ

**Department of Computer Science and Engineering
(Artificial Intelligence and Machine Learning)**

K. RAMAKRISHNAN COLLEGE OF ENGINEERING

**K. RAMAKRISHNAN COLLEGE OF ENGINEERING
(AUTONOMOUS)**

ANNA UNIVERSITY, CHENNAI





**K. RAMAKRISHNAN COLLEGE OF ENGINEERING
(AUTONOMOUS)
ANNA UNIVERSITY, CHENNAI**



BONAFIDE CERTIFICATE

Certified that this project report titled **“MINI GITHUB”** is the bonafide work of **BALAMANIKANDAN S (8115U24AM013)** who carried out the work under my supervision.

SIGNATURE

Dr. B. KIRAN BALA M.E.,M.B.A.,Ph.D.,

**HEAD OF THE DEPARTMENT
ASSOCIATE PROFESSOR,**

Department of Artificial Intelligence
and Machine Learning,
K. Ramakrishnan College of
Engineering, (Autonomous)
Samayapuram, Trichy.

SIGNATURE

Mr. P. NAGARAJ M.E.,

**SUPERVISOR
ASSISTANT PROFESSOR,**

Department of Artificial Intelligence
and Machine Learning,
K. Ramakrishnan College of
Engineering, (Autonomous)
Samayapuram, Trichy.

SIGNATURE OF INTERNAL EXAMINER

NAME:

DATE:

SIGNATURE OF EXTERNAL EXAMINER

NAME:

DATE:



**K. RAMAKRISHNAN COLLEGE OF ENGINEERING
(AUTONOMOUS)
ANNA UNIVERSITY, CHENNAI**



DECLARATION BY THE CANDIDATE

I declare that to the best of my knowledge the work reported here in has been composed solely by myself and that it has not been in whole or in part in any previous application for a degree.

Submitted for the project Viva-Voice held at K. Ramakrishnan College of Engineering on _____

SIGNATURE OF THE CANDIDATE

ACKNOWLEDGEMENT

I thank the almighty GOD, without whom it would not have been possible for me to complete my project.

I wish to address my profound gratitude to **Dr.K.RAMAKRISHNAN**, Chairman, K. Ramakrishnan College of Engineering(Autonomous), who encouraged and gave me all help throughout the course.

I extend my hearty gratitude and thanks to my honorable and grateful Executive Director **Dr.S.KUPPUSAMY, B.Sc., MBA., Ph.D.**, K. Ramakrishnan College of Engineering(Autonomous).

I am glad to thank my Principal **Dr.D.SRINIVASAN, M.E., Ph.D.,FIE., MIIW., MISTE., MISAE., C.Engg**, for giving me permission to carry out this project.

I wish to convey my sincere thanks to **Dr.B.KIRAN BALA, M.E., M.B.A., Ph.D.**, Head of the Department, Artificial Intelligence and Machine Learning for giving me constant encouragement and advice throughout the course.

I am grateful to **Mr.P.NAGARAJ, M.E., Assistant Professor**, Artificial Intelligence and Machine Learning, K. Ramakrishnan College of Engineering (Autonomous), for his guidance and valuable suggestions during the course of study.

Finally, I sincerely acknowledged in no less terms all my staff members, my parents and, friends for their co-operation and help at various stages of this project work.

BALAMANIKANDAN S (8115U24AM013)



K. RAMAKRISHNAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

ANNA UNIVERSITY, CHENNAI



DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

VISION

To become a renowned hub for AIML technologies to producing highly talented globally recognizable technocrats to meet industrial needs and societal expectation.

MISSION

Mission of the Department

- M1** To impart advanced education in AI and Machine Learning, built upon a foundation in Computer Science and Engineering.
- M2** To foster Experiential learning equips students with engineering skills to tackle real-world problems.
- M3** To promote collaborative innovation in AI, machine learning, and related research and development with industries.
- M4** To provide an enjoyable environment for pursuing excellence while upholding strong personal and professional values and ethics.

PROGRAM EDUCATIONAL OBJECTIVES (PEO's)

- PEO1** Excel in technical abilities to build intelligent systems in the fields of AI & ML in order to find new opportunities.
- PEO2** Embrace new technology to solve real-world problems, whether alone or as a team, while prioritizing ethics and societal benefits.
- PEO3** Accept lifelong learning to expand future opportunities in research and product development.

PROGRAM SPECIFIC OUTCOMES (PSO's)

- PSO1** Expertise in tailoring ML algorithms and models to excel in designated applications and fields.
- PSO2** Ability to conduct research, contributing to machine learning advancements and innovations that tackle emerging societal challenges.

PROGRAM OUTCOMES (POs)

Engineering students will be able to:

PO1 Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2 Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3 Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4 Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5 Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6 The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO8 Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings

PO9 Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO10 Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO11 Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

ABSTRACT

The **Mini GiTHub** is a web-based application designed to provide a simplified platform for managing code repositories and understanding the basic concepts of version control systems. The system is inspired by platforms like GitHub but is developed as a lightweight solution suitable for academic and beginner-level use. The application allows users to register, log in, and create repositories to store their project files. Users can upload files, update them, and maintain a record of changes through a commit mechanism. Each modification made to a file is recorded with a message and timestamp, enabling basic version tracking. The system is built using front-end technologies such as HTML, CSS, and JavaScript, while the backend is handled using server-side scripting like PHP or Python. A relational database, MySQL, is used to store structured data such as user information, repository details, files, and commit history. Mini GiTHub demonstrates how database concepts and web technologies can be integrated to build a functional and scalable application. It is particularly useful for students to understand repository management, database design, and basic version control concepts.

ABSTRACT WITH POs AND PSOs MAPPING

ABSTRACT	POs MAPPED	PSOs MAPPED
The Mini GiTHub is a web-based application designed to provide a simplified platform for managing code repositories and understanding the basic concepts of version control systems. The system is inspired by platforms like GitHub but is developed as a lightweight solution suitable for academic and beginner-level use. The application allows users to register, log in, and create repositories to store their project files. Users can upload files, update them, and maintain a record of changes through a commit mechanism. Each modification made to a file is recorded with a message and timestamp, enabling basic version tracking. The system is built using front-end technologies such as HTML, CSS, and JavaScript, while the backend is handled using server-side scripting like PHP or Python. A relational database, MySQL, is used to store structured data such as user information, repository details, files, and commit history. Mini HiTHub demonstrates how database concepts and web technologies can be integrated to build a functional and scalable application. It is particularly useful for students to understand repository management, database design, and basic version control concepts.	PO1 – 3 PO2 – 3 PO3 – 3 PO4 – 3 PO5 – 3 PO6 – 3 PO7 – 3 PO8 – 3 PO9 – 3 PO10 – 3 PO11 – 3	PSO1 – 3 PSO2 – 3

Note: 1- Low, 2-Medium, 3- High

TABLE OF CONTENTS

CHAPTER No.	TITLE	PAGE No.
	ABSTRACT	v
	LIST OF ABBREVIATIONS	x
	LIST OF FIGURES	xi
1	INTRODUCTION	1
	1.1 Objective	1
	1.2 Scope of the Project	2
	1.3 Overview of the System	3
	1.4 Importance of DBMS in Server Monitoring	4
	1.5 Technologies Used	5
2	PROJECT METHODOLOGY	7
	2.1 Requirement Analysis	7
	2.2 Entity-Relationship (ER) Diagram	8
	2.3 Relational Schema	9
	2.4 Normalization Process	9
	2.5 System Architecture Overview	10
3	MODULE DESCRIPTION	12
	3.1 Repository Management Module	12
	3.2 Commit Tracking Module	12
	3.3 Issue Tracker Module	13
	3.4 Pull Request Module	13
	3.5 Search & Filter Module	14
	3.6 Notification Module	14
4	DATABASE IMPLEMENTATION	15
	4.1 Table Creation Scripts	15
	4.2 Constraints and Relationships	17

	4.3	Backup & Recovery Considerations	17
5		RESULTS AND DISCUSSION	19
	5.1	Performance Analysis	19
	5.2	Observations and Interpretations	20
6		CONCLUSION AND FUTURE WORK	21
	6.1	Summary of Outcomes	21
	6.2	Future Scope and Enhancements	22
7		APPENDIX	23
		APPENDIX A – Source Code	23
		APPENDIX B - Screenshots	25
		REFERENCES	27

LIST OF ABBREVIATIONS

S. NO.	ACRONYM	ABBREVIATION
1	API	Application Programming Interface
2	CRUD	Create, Read, Update, Delete
3	DBMS	Database Management System
4	GPS	Global Positioning System
5	HTTP	HyperText Transfer Protocol
6	OSRM	Open-Source Routing Machine
7	SQL	Structured Query Language
8	STT	Speech-to-Text
9	UI	User Interface
10	UX	User Experience

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO.
2.2.1	Entity Relationship Diagram	8
2.3.1	Relational Schema	9
2.5.1	System Architecture	11
B.1	Repositories Table Output	25
B.2	Commits Table Output	25
B.3	Users Table Output	26
B.4	Alerts Table Output	26

CHAPTER 1

INTRODUCTION

1.1 OBJECTIVE

The primary objective of Mini HiTHub is to develop a database-driven web application that enables users to manage their coding projects efficiently.

The specific objectives include:

- To design a platform for creating and managing repositories
- To allow users to upload and manage files within repositories
- To implement a basic version control system using commits
- To demonstrate DBMS concepts such as normalization, relationships, and constraints
- To provide a secure authentication system for users
- To implement CRUD operations across all modules

This project helps in understanding real-world application development using database systems.

The system demonstrates how database management plays a key role in handling structured data such as users, repositories, and file changes. Mini HiTHub is useful for students and beginners to understand version control concepts in a simple way.

1.2 SCOPE OF THE PROJECT

The scope of Mini HiTHub covers both functional and technical aspects of a repository management system.

Functional Scope:

- User registration and login
- Repository creation and management
- File upload, update, and deletion
- Commit history tracking
- Search and filter functionality

Technical Scope:

- Use of MySQL for data storage
- Implementation of relational schema
- Backend processing using server-side scripting
- Responsive user interface

Limitations:

- Does not support real-time collaboration
- No advanced Git features like branching and merging
- Limited to small-scale usage

Despite these limitations, the system is highly effective for learning purposes.

server data, metrics, and alert records securely. DBMS principles such as normalization and referential integrity ensure data consistency and reliability.

1.3 OVERVIEW OF THE SYSTEM

Mini HiTHub is designed as a web-based application that follows a client-server architecture. In this system, users interact with the application through a web browser, where they can register, log in, and access their dashboard. After successful authentication, users can create repositories and upload files associated with those repositories. Each file modification is recorded as a commit, which includes a message and timestamp for tracking purposes. The backend processes user requests and communicates with the database to store and retrieve data. The database maintains structured records of users, repositories, files, and commits, ensuring efficient data handling. The system is designed to be simple, user-friendly, and responsive across different devices.

However, the system does not include advanced features like real-time collaboration, branching, merging, or distributed version control. Despite these limitations, the project serves as a strong foundation for understanding how repository systems work.

1.4 IMPORTANCE OF DBMS IN SERVER MONITORING SYSTEMS

DBMS is a critical component of the Mini HiTHub system. It is responsible for storing, managing, and retrieving all application data efficiently.

Key Roles of DBMS:

- **Data Organization:** Stores user, repository, file, and commit data in structured tables
- **Data Integrity:** Ensures accuracy using constraints like primary key and foreign key
- **Data Security:** Protects sensitive information like passwords

- **Efficient Query Processing:** Retrieves data quickly using SQL queries
- **Relationship Management:** Maintains connections between entities

Without DBMS, managing large amounts of data in this system would be difficult and inefficient.

1.5 TECHNOLOGIES USED

The Mini HiTHub project is developed using a combination of front-end, back-end, and database technologies. The front-end is built using HTML, CSS, and JavaScript, which are responsible for designing the user interface and providing interactivity. The backend is developed using server-side scripting languages such as PHP, Python, or Node.js, which handle business logic, user authentication, and communication with the database. MySQL is used as the database management system to store and manage all application data. Development tools such as Visual Studio Code and XAMPP are used to write, test, and run the application locally. These technologies together ensure that the system is functional, efficient, and scalable..

The **frontend layer** is developed using HTML, CSS, and JavaScript, which are responsible for designing the structure, layout, and interactivity of the web application. HTML is used to create the basic structure of web pages, including forms, buttons, and tables. CSS is used to enhance the appearance of the application by adding colors, layouts, fonts, and responsive design features. JavaScript is used to add dynamic behavior such as form validation, real-time updates, and interactive elements.

Front-End Technologies:

- HTML – Structure of web pages

- CSS – Styling and layout
- JavaScript – Interactivity

Back-End Technologies:

- PHP / Python / Node.js – Server-side processing
- Authentication – Secure login system

Database:

- MySQL – Relational database for storing data

Tools:

- VS Code – Code editor
- XAMPP – Local server environment

CHAPTER 2

LITERATURE SURVEY

This section reviews the project methodology for the Mini GitHub System, which follows a structured and iterative approach aimed at building a reliable, user-friendly, and secure web application. The project is designed using the principles of the Software Development Life Cycle (SDLC), with a focus on requirement gathering, database design, modular implementation, and testing.

2.1 REQUIREMENT ANALYSIS

The first step in developing the Mini GitHub system is to identify both functional and non-functional requirements. These requirements define what the system should do and the constraints under which it should operate.

Functional Requirements:

- User Registration and Login
- Adding, Editing, Deleting, and Viewing Repository Records
- Recording and displaying commits with messages and timestamps
- Issue creation, assignment, and status management
- Pull request creation, review, merge, and close operations
- Admin functionalities: view/delete repositories, users, and records

Non-Functional Requirements:

- Usability: Simple and user-friendly interface
- Performance: Fast data retrieval and minimal load time
- Security: Password encryption, session control, and SQL injection protection
- Scalability: Easily extendable to include more repositories and features
- Maintainability: Code is modular and well-documented

2.2 ENTITY-RELATIONSHIP (ER) DIAGRAM

The ER Diagram illustrates how different entities in the Mini GitHub system are connected. It provides a blueprint for structuring the database.

Main Entities:

- Users: Stores user details like ID, name, email, password, and created timestamp
- Repositories: Contains repository ID, user ID (owner), name, description, privacy flag
- Commits: Includes commit ID, repo ID (foreign key), user ID (foreign key), message, and timestamp
- Issues: Includes issue ID, repo ID (foreign key), user ID (foreign key), title, description, and status
- Pull Requests: Includes PR ID, repo ID (foreign key), user ID (foreign key), title, description, and status

Relationships:

- One repository can have multiple commits (1-to-many)
- One repository can have multiple issues (1-to-many)
- One repository can have multiple pull requests (1-to-many)
- One user can own multiple repositories (1-to-many)
- The project is designed using the principles of the Software Development Life Cycle (SDLC), with a focus on requirement gathering, database design, modular implementation, and testing.

The first step in developing the Mini GitHub system is to identify both functional and non-functional requirements. These requirements define what the system should do and the constraints under which it should operate.

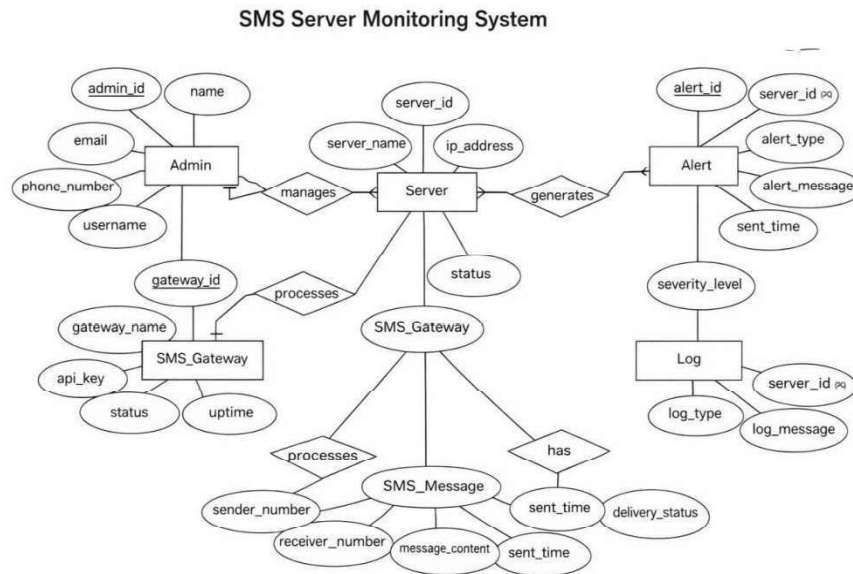


Figure 2.2.1 Entity Relationship

2.3 RELATIONAL SCHEMA

Based on the ER diagram, the relational schema outlines how data is structured into tables and how they relate through keys.

Schema Example:

- users(id, name, email, password, created_at) → id is Primary Key
- repositories(id, user_id, name, description, is_private, created_at) → id is Primary Key, user_id is Foreign Key
- commits(id, repo_id, user_id, message, committed_at) → id is Primary Key, repo_id and user_id are Foreign Keys
- issues(id, repo_id, user_id, title, description, status, created_at) → id is Primary Key, repo_id and user_id are Foreign Keys
- pull_requests(id, repo_id, user_id, title, description, status, created_at) → id is Primary Key, repo_id and user_id are Foreign Keys

This schema is normalized to reduce redundancy and ensure data integrity.

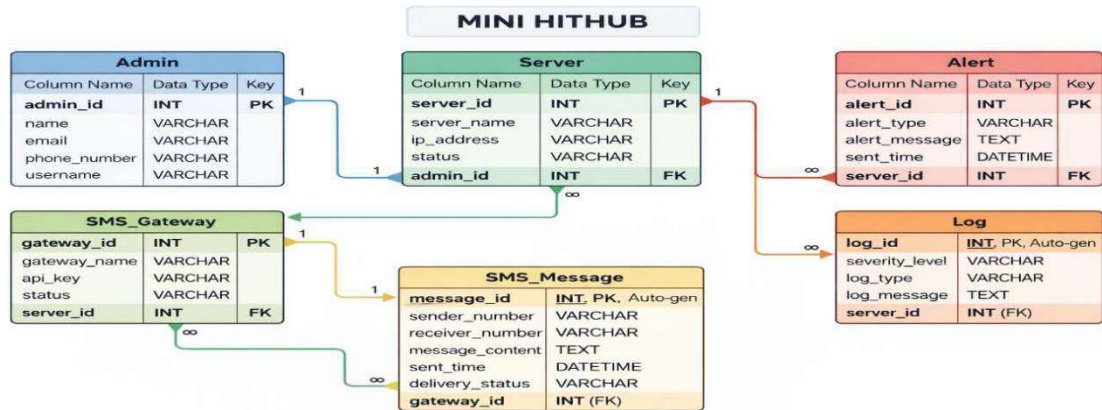


Figure 2.3.1 Relational Schema

2.4 NORMALIZATION PROCESS

Normalization is the process of organizing data in the database to reduce duplication and ensure efficient access. The Mini GitHub platform applies the first three normal forms:

- 1NF (First Normal Form): Ensures atomicity of data (no repeating groups or arrays in a single column)
- 2NF (Second Normal Form): Ensures that all non-key attributes are fully dependent on the primary key
- 3NF (Third Normal Form): Eliminates transitive dependency between non-key attributes

This leads to a clean, efficient, and scalable database design.

2.5 SYSTEM ARCHITECTURE OVERVIEW

The architecture of the Mini GitHub System follows a three-tier model:

- Presentation Layer (Frontend): Built using HTML, CSS, and JavaScript. Includes user interfaces for login, dashboard, repository listing, commit view, issue tracker, and pull request log.

- **Application Layer (Backend):** Developed using PHP, Python, or Node.js. Processes user input, handles business logic, communicates with the database, and manages repository operations.
- **Data Layer (Database):** Managed using MySQL. Stores all persistent data including user information, repository records, commit logs, issues, and pull requests.

The system uses RESTful API routes to handle requests such as:

- GET /repositories – fetch list of all repositories
- POST /login – authenticate user
- POST /commit – create new commit record
- DELETE /repository/{id} – remove repository (admin only)

Security features like password hashing, session tokens, and input validation are included across all layers.

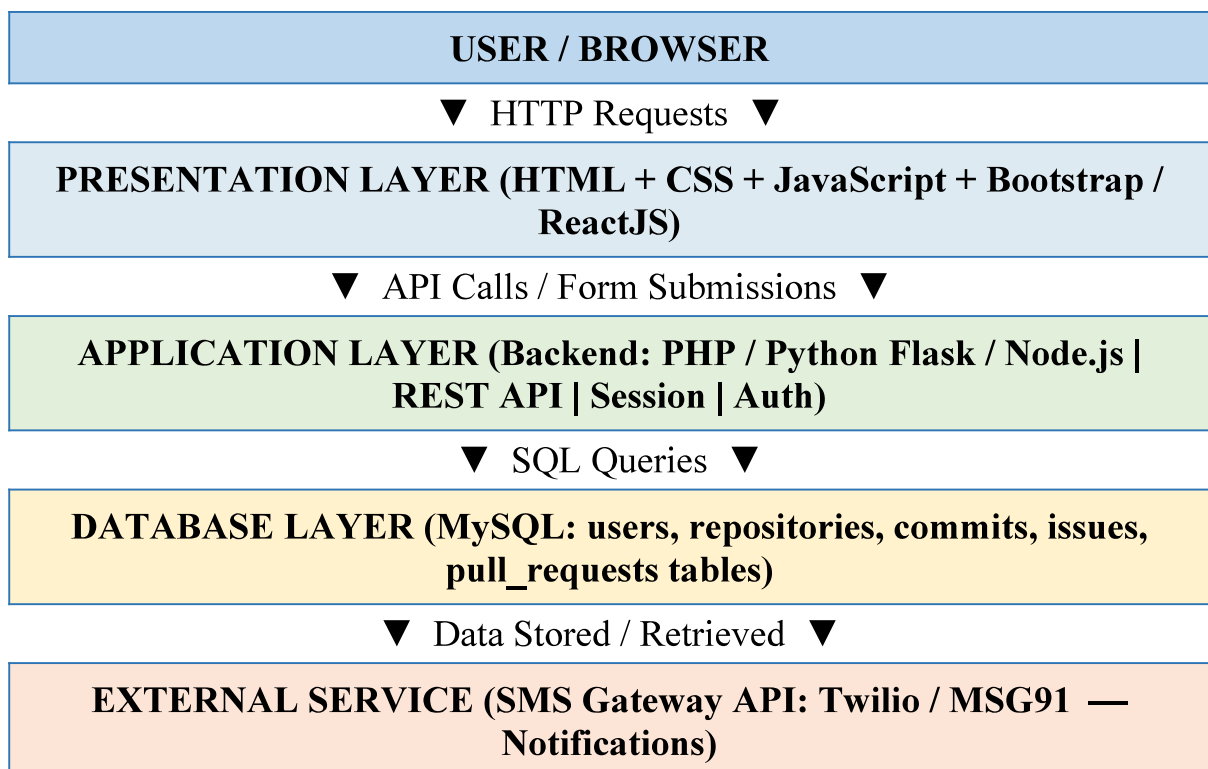


Figure 2.5.1 System Architecture

CHAPTER 3

MODULE DESCRIPTION

The Mini GitHub System is structured as a modular web application where each component is responsible for a specific functionality. Adopting a modular design helps in achieving separation of concerns, maintainability, and scalability.

3.1 REPOSITORY MANAGEMENT MODULE

The Repository Management Module handles all functionalities related to repository creation and configuration. It defines which repositories are tracked and provides the administrative interface for managing repository entries.

Key Features:

- **Repository Creation:** Users create new repositories by entering a name, description, and privacy setting stored in the repositories table.
- **Edit & Delete:** Users update repository details or remove decommissioned repositories from the platform.
- **Listing & Viewing:** All repositories are displayed in a clean list view with their current status and associated commits.

Functionality Flow:

- User fills the repository creation form with name and description.
- Input is validated and stored in the repositories table.

Repository list is retrieved and displayed on the user dashboard

3.2 COMMIT TRACKING MODULE

This is the core functional module. It records and displays commit history for each repository including commit messages, author, and timestamps.

Key Features:

- **Commit Recording:** Commit data is inserted into the commits table with a reference to the corresponding repository and user, along with a timestamp.
- **Display:** Commits are retrieved from the database and displayed in tabular format sorted by repository and recording time.
- **Audit Trail:** Commit history serves as a full audit trail of code changes made to each repository.

3.3 ISSUE TRACKER MODULE

This module allows users to raise, track, and resolve issues within a repository. Issues are linked to repositories and can be assigned open or closed status.

Key Features:

- **Issue Creation:** An issue record is created in the issues table linking the user, repository, title, and description.
- **Status Management:** Issues can be marked as open or closed by repository owners.
- **Issue History:** All past issues are stored and can be reviewed by admins and collaborators.
- **Time Stamping:** Each issue is tagged with the exact date and time it was generated.

3.4 PULL REQUEST MODULE

This module manages code review and collaboration through pull requests. Users can submit PRs, have them reviewed, and merge or close them.

Key Features:

- **User Management:** View all users and their contributions; block or remove suspicious accounts.
- **Repository Moderation:** View, edit, or delete repository records.

- Pull Request Management: Review all PR logs, close outdated PRs, and reconfigure repository settings.

3.5 SEARCH & FILTER MODULE

To improve user experience, this module allows users to search for repositories and filter commits or issues based on repository name, user, or date range.

Key Features:

- Keyword Search: Allows users to search by repository name or description.
- Filter Options: Commits and issues can be filtered by type, repository, or date range.
- Dynamic Results: Search results are displayed in real-time without full page reloads.

3.6 NOTIFICATION MODULE

3.6 NOTIFICATION MODULE

This module ensures that users and administrators are kept informed about critical repository events through notification channels.

Key Features:

- Email / In-App Notifications: Automatically alert users when issues are raised or pull requests are submitted on their repositories.
- In-App Alerts: Display notifications in the user dashboard about recent activity in followed repositories.

CHAPTER 4

DATABASE IMPLEMENTATION

A robust and well-structured database is the backbone of the Mini GitHub System. The database plays a critical role in storing, organizing, and retrieving repository records, commit histories, user accounts, issues, and pull requests.

4.1 TABLE CREATION SCRIPTS

The system uses MySQL as its RDBMS. The following tables form the core schema of the Mini GitHub database. Each table is created with appropriate data types, constraints, and relationships to ensure data integrity.

```
-- Create Database
CREATE DATABASE minigithub;
USE minigithub;

-- =====
-- USERS TABLE
-- =====
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO users (name, email, password) VALUES
('Sivakumar', 'siva@gmail.com', 'hashed_pass1'),
('Anitha', 'anitha@gmail.com', 'hashed_pass2'),
('Rajan', 'rajan@gmail.com', 'hashed_pass3');

SELECT * FROM users;

-- =====
-- REPOSITORIES TABLE
```

```

-- =====
CREATE TABLE repositories (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    is_private BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

INSERT INTO repositories (user_id, name, description, is_private)
VALUES
(1, 'mini-github', 'A mini version control system', FALSE),
(2, 'ai-project', 'AI and Data Science project', FALSE),
(3, 'dbms-lab', 'Database management lab work', TRUE),
(1, 'web-portfolio', 'Personal portfolio website', FALSE);

SELECT * FROM repositories;

-- =====
-- COMMITS TABLE
-- =====
CREATE TABLE commits (
    id INT AUTO_INCREMENT PRIMARY KEY,
    repo_id INT NOT NULL,
    user_id INT NOT NULL,
    message VARCHAR(500) NOT NULL,
    committed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (repo_id) REFERENCES repositories(id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

INSERT INTO commits (repo_id, user_id, message) VALUES
(1, 1, 'Initial commit - added README'),
(1, 1, 'Added user authentication module'),
(2, 2, 'Created data preprocessing script'),
(3, 3, 'Added ER diagram for DBMS project'),
(1, 1, 'Fixed login bug and updated UI');

SELECT * FROM commits;

```

```

-- =====
-- ISSUES TABLE
-- =====
CREATE TABLE issues (
    id INT AUTO_INCREMENT PRIMARY KEY,
    repo_id INT NOT NULL,
    user_id INT NOT NULL,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    status ENUM('open', 'closed') DEFAULT 'open',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (repo_id) REFERENCES repositories(id) ON DELETE
    CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

INSERT INTO issues (repo_id, user_id, title, description, status)
VALUES
(1, 2, 'Login page not responsive', 'The login page breaks on
mobile screens', 'open'),
(1, 3, 'Add dark mode support', 'Users requested dark mode for
better UX', 'open'),
(2, 1, 'Data pipeline error', 'Script fails when data has null
values', 'closed'),
(3, 2, 'Update schema diagram', 'Schema needs to reflect new
tables', 'open');

SELECT * FROM issues;

-- =====
-- PULL REQUESTS TABLE
-- =====
CREATE TABLE pull_requests (
    id INT AUTO_INCREMENT PRIMARY KEY,
    repo_id INT NOT NULL,
    user_id INT NOT NULL,
    title VARCHAR(255) NOT NULL,
    description TEXT,
    status ENUM('open', 'merged', 'closed') DEFAULT 'open',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (repo_id) REFERENCES repositories(id) ON DELETE
    CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

```

```

INSERT INTO pull_requests (repo_id, user_id, title, description,
status) VALUES
(1, 2, 'Fix responsive layout', 'Fixes the layout issue reported in
issue #1', 'open'),
(1, 3, 'Add dark mode CSS', 'Implements dark mode styles for all
pages', 'merged'),
(2, 1, 'Null check in pipeline', 'Adds null validation before data
processing', 'closed');

SELECT * FROM pull_requests;

-- Do NOT drop database unless needed
-- DROP DATABASE minigithub;

```

4.2 CONSTRAINTS AND RELATIONSHIPS

Implementing primary keys, foreign keys, and constraints ensures that the database maintains data integrity.

- **Primary Keys:** Uniquely identify each record (id in all tables).
- **Foreign Keys:** repo_id in commits, issues, and pull_requests links to repositories; user_id in all child tables links to users.
- **NOT NULL:** Essential fields like repository name, commit message, and issue title must always be provided.
- **ON DELETE CASCADE:** When a repository or user is deleted, all related records are automatically removed.

4.3 BACKUP & RECOVERY CONSIDERATIONS

Ensuring data safety and reliability is essential. The system includes backup and recovery strategies to prevent data loss.

- **Automated Daily Backups:** Full database dumps using mysqldump or server-based backup services.
- **Cloud Storage Integration:** Backups stored in Google Drive, AWS S3, or Dropbox.

- Point-in-Time Recovery: Using MySQL binary logs to restore the database to a specific time before data corruption.

CHAPTER 5

RESULTS AND DISCUSSION

After successful development and implementation of the Mini GitHub System, the system was thoroughly tested to evaluate its functionality, performance, and user experience.

5.1 PERFORMANCE ANALYSIS

The performance of the system was assessed across frontend responsiveness, database operations, and system reliability.

Frontend Performance:

- The web interface was responsive across different devices (desktop, tablet, mobile).
- Page load times were under 2 seconds in local testing.
- Form validation and navigation were smooth, offering a user-friendly experience.

Backend Processing:

- CRUD operations on repository records, commit logs, issues, and pull requests executed reliably.
- Login and registration worked with password encryption and session handling.
- Repository and commit operations were handled effectively with minimal server delay.

Database Operations:

- SQL queries returned results efficiently due to well-structured schema and indexing.
- Foreign keys and cascading rules maintained relational integrity throughout testing.

Security Measures:

- Passwords were encrypted using bcrypt hashing algorithms.
- Input validation and prepared statements prevented SQL injection attacks.
- Session timeouts and logout functions protected user access.

5.2 OBSERVATIONS AND INTERPRETATIONS**User Experience:**

- The platform was intuitive and simple to navigate.
- Developers appreciated the ability to manage all repositories and commits from a single dashboard.
- The issue tracker and pull request system provided immediate awareness of collaboration tasks.

Admin Insights:

- Admins could effectively monitor repository records, commit history, and issue logs.
- The dashboard was easy to use but could benefit from graphical commit trend charts.

Functional Stability:

- All major modules worked as expected with no crashes or unexpected behavior.
- Error handling was implemented on both client and server sides.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 SUMMARY OF OUTCOMES

The Mini GitHub System was successfully designed, developed, and tested as a functional web application backed by a normalized relational database. The project demonstrated the effective integration of a DBMS with a version control interface to deliver real-time repository and collaboration management.

Key Findings:

- **Database Design:** The normalized schema with five core tables effectively captured all necessary relationships and enforced referential integrity through foreign keys.
- **CRUD Operations:** All insert, select, update, and delete operations performed correctly with proper cascade behavior.
- **Collaboration Features:** The issue tracker and pull request module accurately logged collaboration activities and provided a framework for team-based development workflows.
- **Web Interface:** The system delivered a clean and functional user interface accessible across multiple devices.

Limitations:

- **Real-time code diff viewing** requires integration with system-level diff utilities.
- The current implementation does not include graphical dashboards for commit trend visualization.
- Branch management and merge conflict resolution are not yet implemented.

6.2 FUTURE SCOPE AND ENHANCEMENTS

Proposed Future Enhancements:

- **Graphical Dashboards:** Integration of Chart.js for visual representation of commit frequency, issue resolution rates, and pull request trends over time.
- **Branching Support:** Implement branch creation, switching, and merge operations with conflict detection.
- **Code Diff Viewer:** Deploy side-by-side diff viewing for commits and pull requests.
- **Multi-Channel Notifications:** Extend alerting to include email, WhatsApp, or Slack alongside in-app notifications.
- **Predictive Analytics:** Use historical commit data and machine learning to predict development bottlenecks.
- **Mobile App Version:** Extend the platform as a mobile app using React Native or Flutter.

The flexible structure of the existing application supports the integration of these features with minimal architectural changes. This project demonstrated the importance of modular design, database-driven architecture, and full-stack development in building scalable, real-world version control applications.

APPENDICES

APPENDIX A

SOURCE CODE (SQL)

This section presents the complete SQL source code for the SMS Based Remote Server Monitoring System. The code creates the database, defines all tables with appropriate constraints and relationships, and populates them with sample data.

A.1 Complete Database Schema and Sample Data

```
-- Create Database
CREATE DATABASE sms;
USE sms;
-- =====
-- SERVERS TABLE
-- =====
CREATE TABLE servers (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    ip_address VARCHAR(255) NOT NULL
);
INSERT INTO servers (name, ip_address) VALUES
('Server 1', '192.168.1.100'),
('Server 2', '192.168.1.101'),
('Server 3', '192.168.1.102'),
('Server 4', '192.168.1.103');

SELECT * FROM servers;
-- =====
-- METRICS TABLE
-- =====
CREATE TABLE metrics (
    id INT AUTO_INCREMENT PRIMARY KEY,
    server_id INT NOT NULL,
    metric_type VARCHAR(50) NOT NULL,
    value FLOAT NOT NULL,
    recorded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (server_id) REFERENCES servers(id)
    ON DELETE CASCADE
);

INSERT INTO metrics (server_id, metric_type, value) VALUES
(1, 'CPU Usage', 30.50),
(2, 'CPU Usage', 40.20),
(3, 'CPU Usage', 10.30),
```

```

(4, 'CPU Usage', 11.00),
(1, 'Memory Usage', 50.00),
(2, 'Memory Usage', 60.00),
(3, 'Memory Usage', 70.00),
(4, 'Memory Usage', 80.00),
(1, 'Disk Usage', 75.00),
(2, 'Disk Usage', 80.00),
(3, 'Disk Usage', 85.00),
(4, 'Disk Usage', 90.00);

SELECT * FROM metrics;
-- =====
-- USERS TABLE
-- =====
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL,
    phone_number VARCHAR(20) NOT NULL
);

INSERT INTO users (name, email, phone_number) VALUES
('John', 'john@gmail.com', '1234567890'),
('Sarah', 'sarah@gmail.com', '9876543210'),
('Mike', 'mike@gmail.com', '5551234567');

SELECT * FROM users;
-- =====
-- ALERTS TABLE
-- =====
CREATE TABLE alerts (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    server_id INT NOT NULL,
    message VARCHAR(255) NOT NULL,
    alert_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
        ON DELETE CASCADE,
    FOREIGN KEY (server_id) REFERENCES servers(id)
        ON DELETE CASCADE
);

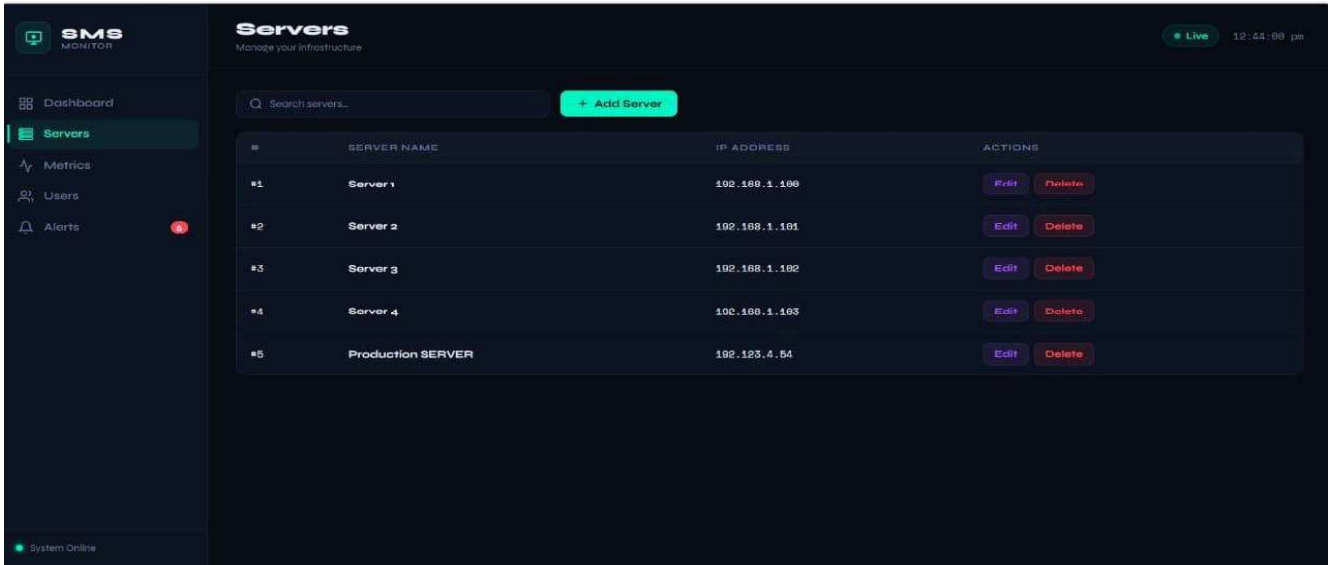
INSERT INTO alerts (user_id, server_id, message) VALUES
(1, 3, 'Disk space is running low on Server 3'),
(2, 2, 'High CPU usage detected on Server 2'),
(3, 4, 'Server 4 is down!'),
(1, 1, 'Network traffic is unusually high on Server 1'),
(2, 3, 'Memory usage is approaching critical on Server 3');

SELECT * FROM alerts;
-- Do NOT drop database unless needed
-- DROP DATABASE sms;

```

APPENDIX B – Screenshots

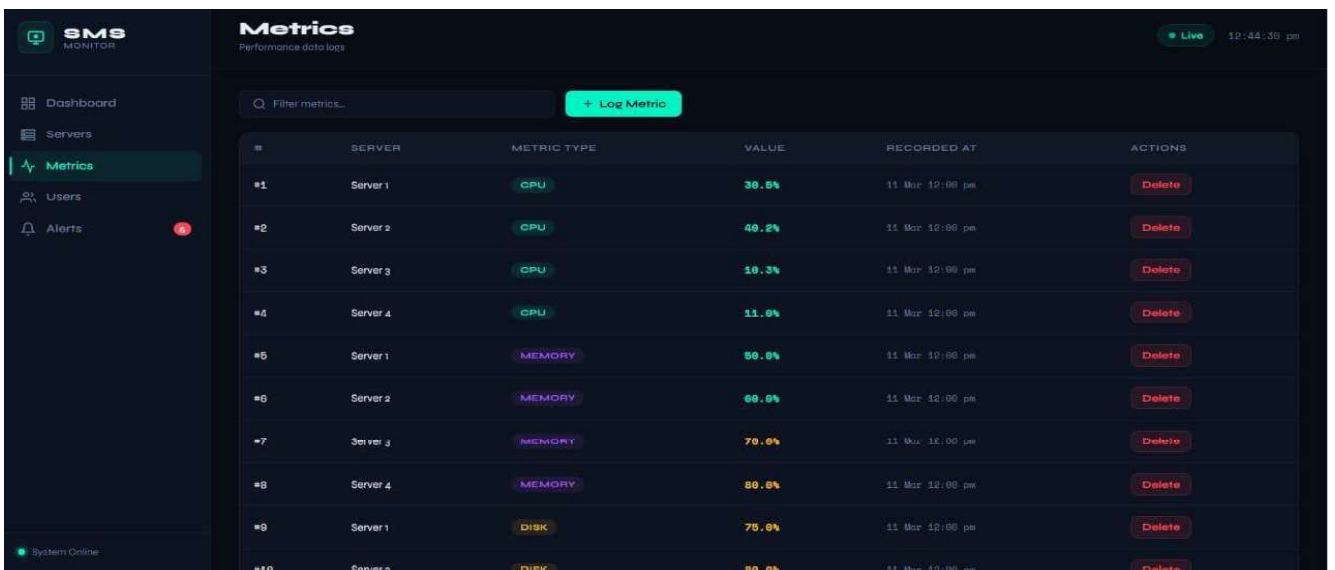
The following screenshots illustrate the key outputs obtained after executing the SQL scripts in MySQL Workbench.



The screenshot shows the 'Servers' page in the SMS MONITOR application. The page has a dark theme and a sidebar on the left with navigation links: Dashboard, Servers (active), Metrics, Users, and Alerts. The main content area is titled 'Servers' and includes a search bar and an '+ Add Server' button. Below this is a table with the following data:

#	SERVER NAME	IP ADDRESS	ACTIONS
#1	Server 1	192.168.1.100	Edit Delete
#2	Server 2	192.168.1.101	Edit Delete
#3	Server 3	192.168.1.102	Edit Delete
#4	Server 4	192.168.1.103	Edit Delete
#5	Production SERVER	192.123.4.54	Edit Delete

Figure B.1 Servers Table Output



The screenshot shows the 'Metrics' page in the SMS MONITOR application. The page has a dark theme and a sidebar on the left with navigation links: Dashboard, Servers, Metrics (active), Users, and Alerts. The main content area is titled 'Metrics' and includes a search bar and a '+ Log Metric' button. Below this is a table with the following data:

#	SERVER	METRIC TYPE	VALUE	RECORDED AT	ACTIONS
#1	Server 1	CPU	38.5%	11 Mar 12:00 pm	Delete
#2	Server 2	CPU	48.2%	11 Mar 12:00 pm	Delete
#3	Server 3	CPU	10.3%	11 Mar 12:00 pm	Delete
#4	Server 4	CPU	11.0%	11 Mar 12:00 pm	Delete
#5	Server 1	MEMORY	50.0%	11 Mar 12:00 pm	Delete
#6	Server 2	MEMORY	60.0%	11 Mar 12:00 pm	Delete
#7	Server 3	MEMORY	70.0%	11 Mar 12:00 pm	Delete
#8	Server 4	MEMORY	80.0%	11 Mar 12:00 pm	Delete
#9	Server 1	DISK	75.0%	11 Mar 12:00 pm	Delete
#10	Server 2	DISK	85.0%	11 Mar 12:00 pm	Delete

Figure B.2 Metrics Table Output

SMS MONITOR

Users
Registered system users

● Live 12:44:53 pm

Search users... + Add User

#	NAME	EMAIL	PHONE	ACTIONS
#1	John	john@gmail.com	1234567890	Edit Delete
#2	Sarah	sarah@gmail.com	9876543210	Edit Delete
#3	Mike	mike@gmail.com	5551234567	Edit Delete

System Online

Figure B.3 Users Table Output

SMS MONITOR

Alerts
System notifications & warnings

● Live 12:46:16 pm

Search alerts... + New Alert

#	USER	SERVER	MESSAGE	TIME	ACTIONS
#1	John	Server 3	Disk space is running low on Server 3	11 Mar 12:00 pm	Delete
#4	John	Server 1	Network traffic is unusually high on Server 1	11 Mar 12:00 pm	Delete
#2	Sarah	Server 2	High CPU usage detected on Server 2	11 Mar 12:00 pm	Delete
#6	Sarah	Server 3	Memory usage is approaching critical levels on Server 3	11 Mar 12:00 pm	Delete
#3	Mike	Server 4	Server 4 is down!	11 Mar 12:00 pm	Delete

System Online

Figure B.4 Alerts Table Output

REFERENCES

1. Bootstrap Documentation. (2024). Responsive Design with Bootstrap. Retrieved from <https://getbootstrap.com/docs/5.3/getting-started/introduction/>
2. GeeksforGeeks. (2024). CRUD Operations in PHP with MySQL. Retrieved from <https://www.geeksforgeeks.org/php-crud-operations/>
3. GitHub. (2024). GitHub Documentation – Managing Repositories. Retrieved from <https://docs.github.com/>
4. Mozilla Developer Network (MDN). (2024). HTML, CSS, and JavaScript Documentation. Retrieved from <https://developer.mozilla.org/>
5. MySQL Official Documentation. (2024). MySQL 8.0 Reference Manual. Retrieved from <https://dev.mysql.com/doc/>
6. PHP Manual. (2024). password_hash() and password_verify() Functions. Retrieved from <https://www.php.net/manual/en/function.password-hash.php>
7. TutorialsPoint. (2024). DBMS – Normalization. Retrieved from https://www.tutorialspoint.com/dbms/dbms_normalization.htm
8. W3Schools. (2024). PHP MySQL Database. Retrieved from https://www.w3schools.com/php/php_mysql_intro.asp
9. Stack Overflow. (2024). Community Discussions on PHP and MySQL Integration. Retrieved from <https://stackoverflow.com/>
10. Pro Git. (2024). Git – Version Control Concepts. Retrieved from <https://git-scm.com/book/en/v2>